

CS 61A Midterm 2 Study Guide

Values that are false in a boolean context:

- 0 and 0.0
- False and None
- An empty container, such as '', [], {}, {}

Others are true values.

Membership

```
>>> digits = [1, 8, 2, 8]
>>> 2 in digits
True
>>> 1828 not in digits
True
```

```
>>> [1, 8] in digits
False
>>> 'd' in 'hotdog'
True
>>> 'dog' in 'hotdog'
True
```

Error Types

IndexError	indexing a list past its last element
KeyError	a key isn't in a dictionary
TypeError	an operation on the wrong type: calling a function with wrong number of arguments; indexing or calling None
AttributeError	an attribute is not found for an object
NameError	A name is not bound in the environment

Lambda & Currying: e.g. $f(x, y) \rightarrow f(x)(y)$

```
>>> square = lambda x: x * x # anonymous function
>>> curry2 = lambda f: lambda x: lambda y: f(x, y)
>>> square(4), curry2(add)(3)(4)
(16, 7)
```

Function Inputs, Outputs, & Type Hints

Always ask: what type goes in, what type comes out?

```
def sum_list(s: list[int]) -> int:
    ...
```

Python doesn't check or enforce type hints. They document a function's intended behavior.

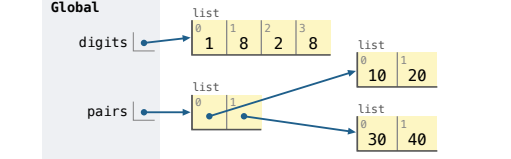
list[int]	list of ints	None	no return
dict[str,int]	str → int	A B	an A or a B
tuple[int,str]	(int,str) pair	tuple[()]	()
Link[int]	T = int	LinkedList	Link or ()

Type variables: A name, usually T, is used to indicate that the same type is involved in two or more formal parameters or return values.

```
def pair[T](a: T, b: T) -> list[T]:
    return [a, b]
```

Lists

```
>>> digits = [1, 8, 2, 8]
>>> len(digits)
4
>>> digits[3]
8
>>> digits[-1] # last
8
>>> [2, 7] + digits * 2
[2, 7, 1, 8, 2, 8, 1, 8, 2, 8]
```



Slicing: `s[<start>:<end>]` makes a new list: from `<start>` (included) up to `<end>` (not included). Defaults: `start=0`, `end=len(s)`. `s[:]` and `list(s)` copy the whole list.

```
>>> s = [6, 5, 4, 3, 2, 1, 0]
>>> s[1:3]
[5, 4]
>>> s[3:]
[6, 5, 4]
>>> s[3:1]
[3, 2, 1, 0]
>>> s[-3:] #last three elements
[2, 1, 0]
```

```
>>> s[:]
[6, 5, 4, 3, 2, 1, 0]
>>> s[10:]
[]
>>> empty = []
>>> empty[0]
IndexError
>>> empty[1:]
[]
```

A slice of a list always evaluates to a list (maybe empty) and never causes an IndexError.

Iteration: for Statements

```
for <name> in <expression>:
    <suite>
```

1. Evaluate the header `<expression>`, which must be an iterable value (a list, tuple, range, dict, etc.)

2. For each element in that iterable value, in order:

- Bind `<name>` to that element in the current frame
- Execute the `<suite>`

Unpacking in a for statement:

```
>>> pairs = [[1, 2], [2, 2],
            [3, 2], [4, 4]]
>>> same_count = 0
>>> for x,y in pairs:
...     if x == y:
...         same_count += 1
>>> same_count
2
```

Annotations:

- `pairs` is a sequence of fixed-length sequences
- A name for each element in a fixed-length sequence

Ranges

`range(-2, 2)` contains ..., -3, -2, -1, 0, 1, 2, 3, ...

- Length: ending value - starting value.
- Element selection: starting value + index.

```
>>> r = range(-2, 2)
>>> r
range(-2, 2)
>>> len(r)
4
```

```
>>> list(range(-2, 2))
[-2, -1, 0, 1]
>>> list(range(4))
[0, 1, 2, 3]
```

Aggregation Functions for an iterable s

<code>sum(s, start=0)</code>	total
<code>max(s, key=None), max(a, b, ..., key=None)</code>	largest
<code>min(s, key=None), min(a, b, ..., key=None)</code>	smallest
<code>all(s)</code>	all true?
<code>any(s)</code>	any true?
<code>len(s)</code>	length

```
>>> all([False, True])
False
>>> all([])
True
>>> sum([1, 2])
3
>>> sum([1, 2], 3)
6
>>> sum([])
0
>>> sum([1], [2], [])
[1, 2]
>>> any([False, True])
True
>>> any([])
False
>>> max(1, 2)
2
>>> max([1, 2])
2
>>> max([1, -2], key=abs)
-2
>>> len([1, 2], 3)
2
```

Dictionaries

A container of key-value pairs; keys must be unique; iteration gives the keys; `d[k]` errors on a missing key; `d.get(k, default)` returns default if `k` is not a key.

```
>>> words = {'más': 'more', 'otro': 'other', 'agua': 'water'}
>>> len(words)
3
>>> "agua" in words
True
>>> "water" in words
False
>>> "water" in words.values()
True
```

```
>>> words["otro"]
'other'
>>> words["hola"]
KeyError
>>> words.get("otro", "hi")
'other'
>>> words.get("hola", "hi")
'hi'
>>> list(words)
['más', 'otro', 'agua']
>>> list(words.values())
['more', 'other', 'water']
```

List & Dictionary Comprehensions

`[<map exp> for <name> in <iter exp> if <filter exp>]`

Short version: `[<map exp> for <name> in <iter exp>]`

A combined expression that evaluates to a list by:

1. For each item in the iterable value of `<iter exp>`, bind `<name>` to that item, then if `<filter exp>` evaluates to a true value, include the value of `<map exp>` in the resulting list
2. `<name>` is not bound in the current frame.

```
>>> [word for word in words if len(word) == 4]
['otro', 'agua']
>>> [word for word in words]
['más', 'otro', 'agua']
>>> [words[word] for word in words]
['more', 'other', 'water']
>>> {x: x*x for x in range(3,6)} # dict comprehension
{3: 9, 4: 16, 5: 25}
>>> x
NameError
```

Tuples

```
>>> empty = ()
>>> len(empty)
0
>>> print(empty)
()
>>> (2, 3) + (4, 5)
(2, 3, 4, 5)
```

```
>>> conditions = ('rain', 'shine')
>>> conditions[0]
'rain'
>>> ra, sh = conditions
>>> sh
'shine'
```

Classes with @dataclass

A `class` statement creates a new type. When decorated with `@dataclass`, each name that has a type hint becomes an attribute of the instances of this new class. The `__str__` method within the class determines the output when `str` or `print` is called on an instance.

```
@dataclass
class Position:
    "A geographic position."
    lat: float
    lon: float

    def __str__(self):
        return format_pos(self)

def format_pos(p: Position) -> str:
    "Format the position with directional suffixes."
    assert abs(p.lat) <= 90 and abs(p.lon) <= 180, \
        f'p{r} is not on Earth!'
    ... # formats a Position as 33.87° S, 151.21° E

>>> sydney = Position(-33.87, 151.21)
>>> sydney.lat
-33.87
>>> type(sydney) == Position
True
>>> print(sydney)
33.87° S, 151.21° E
```

Recursion

State what the function takes in and should return.

`sum_digits(n)`: takes a positive `int` `n`; returns the sum of its digits.

- Base case: an input simple/small enough that the return value can be found without recursion. One digit: return `n`.
- Recursive case: call on a simpler/smaller input and rely on the recursive call's correctness: `sum_digits(all_but_last)` returns the digit sum of all but the last digit; add last.

```
def sum_digits(n: int) -> int:
    "Sum the digits of positive integer n."
    if n < 10:
        return n
    else:
        all_but_last, last = n // 10, n % 10
        return sum_digits(all_but_last) + last
```

Annotations:

- def header: like any function
- Base: no recursive calls
- Conditionals: base cases
- Recursive: recursive calls

Tree Recursion

A recursive case makes more than one recursive call.

```
def fib(n: int) -> int:
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-2) + fib(n-1)
```

n	fib(n)	n	fib(n)
0	0	5	5
1	1	6	8
2	1	7	13
3	2	8	21
4	3		

Recursive decomposition: find simpler instances of the same problem that complete most of the work.

`count_partitions(n, m)` = ways to write `n` as a sum of parts up to size `m`. For (6, 4), explore two possibilities:

- use at least one 4: what's left is a partition of 2 - `count_partitions(2, 4)`
- don't use any 4: parts up to 3 - `count_partitions(6, 3)`

The last step of the recursive case is to add the two counts to get a total count for (6, 4).

```
def count_partitions(n: int, m: int) -> int:
    if n == 0:
        return 1
    elif n < 0:
        return 0
    elif m == 0:
        return 0
    else:
        with_m = count_partitions(n-m, m)
        without_m = count_partitions(n, m-1)
        return with_m + without_m
```

Linked Lists

A linked list is either empty `()` or a `Link` with a `first` value and the `rest` of the list.

- An empty list is `()`. Test if `s` is empty with `not isinstance(s, Link)` or `s == ()`.
- There's no indexing: to reach item `k`, follow `.rest` `k` times.

`type LinkedList[T] = Link[T] | tuple[()]`

```
@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = () # empty list

    def __str__(self):
        return format_link(self)

def format_link(s: Link) -> str:
    """Return a Link s formatted as items within parentheses."""
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), f'{s!r} is not a LinkedList'
    return string + ')'
```

Examples

```
def len_link(s: LinkedList) -> int:
    """Return the length of a linked list.

    >>> len_link(Link(3, Link(4, Link(5))))
    3
    """
    length = 0
    while isinstance(s, Link):
        length += 1
        s = s.rest
    return length

def len_link(s: LinkedList) -> int:
    if not isinstance(s, Link):
        return 0
    return 1 + len_link(s.rest)

def range_link(start: int, end: int) -> LinkedList[int]:
    if start >= end:
        return ()
    else:
        return Link(start, range_link(start + 1, end))
```

Annotations:

- Stops when `s` is `()`
- Base case: `s` is `()`